

ROS 2 基础深度 Demo 总结

1. 本次 Demo 的目标

本次 Demo 的目标不是显示深度图，而是编写一个属于自己的 ROS 2 Python 节点，读取 Gemini Max 发布的深度数据，并计算图像中心区域对应的实际距离。

最终实现的数据链路如下：



本次实际输出约为：

```
Center=(320, 240), valid_pixels=121, distance=965 mm (0.965 m)
```

这证明以下环节均已正常工作：

1. 相机可以采集深度数据。
2. Orbbec 驱动可以把深度数据发布到 ROS 2 话题。
3. 自己编写的节点可以订阅该话题。
4. `CvBridge` 可以将 ROS 图像消息转换为 NumPy 数组。
5. 程序可以过滤无效深度并计算真实距离。

2. 已知运行环境

- 计算平台：Jetson Orin Nano Super 4GB
- 操作系统：Ubuntu 22.04
- ROS 2：Humble
- 相机：Orbbec Gemini Max
- ROS 2 工作空间： `~/orbbec_ros2_ws`
- 相机驱动包： `orbbec_camera`
- Demo 功能包： `depth_demo`
- Demo 节点名称： `depth_reader`
- Demo 可执行程序名称： `depth_reader`

- 深度话题: `/camera/depth/image_raw`
- 深度消息类型: `sensor_msgs/msg/Image`
- 深度编码: `16UC1`
- 深度单位: 毫米

这里有三个容易混淆的名称:

名称	本次使用值	作用
功能包名	<code>depth_demo</code>	组织代码并供 ROS 2 查找
可执行程序名	<code>depth_reader</code>	<code>ros2 run</code> 启动时使用
节点名	<code>/depth_reader</code>	节点进入 ROS 2 通信图后的名称

3. 为什么相机节点和 Demo 节点要分别运行

Orbbec 相机驱动节点负责操作硬件并发布数据, 它是数据的生产者。 `depth_reader` 负责订阅并处理数据, 它是数据的消费者。

二者职责不同:

- 相机驱动: 读取硬件 → 发布深度图
- `depth_reader`: 订阅深度图 → 计算中心距离
- `rqt_image_view`: 订阅深度图 → 将数据画到窗口中

因此, 启动相机驱动并不等于启动显示工具, 也不等于启动自己编写的算法节点。ROS 2 允许多个节点同时订阅一个话题, 彼此通常不会冲突。

实际使用时通常需要两个终端:

- 终端 1: 运行相机驱动, 持续发布深度图。
- 终端 2: 运行 `depth_reader`, 持续处理深度图。

4. ROS 2 命令的基本格式

ROS 2 CLI 的总体形式为:

```
ros2 <命令组> <具体命令> [参数]
```

例如:

```
ros2 topic list
```

其中:

- `ros2`: ROS 2 命令行入口。
- `topic`: 操作话题的命令组。
- `list`: 列出当前发现的话题。

4.1 加载 ROS 2 环境

每打开一个新终端，都应执行：

```
source /opt/ros/humble/setup.bash
source ~/orbbeo_ros2_ws/install/setup.bash
```

第一条加载系统安装的 ROS 2 Humble，第二条加载自己工作空间中编译出来的功能包。

如果没有加载工作空间环境，常见结果是 ROS 2 找不到 `orbbeo_camera`、`depth_demo` 或其中的可执行程序。

4.2 启动相机驱动

`ros2 launch` 的标准格式是：

```
ros2 launch <功能包名> <launch文件名> [参数名:=参数值]
```

只启动 Gemini Max 深度流时，可以使用：

```
ros2 launch orbbeo_camera ob_camera.launch.py \
  enable_color:=false \
  enable_depth:=true \
  depth_width:=640 \
  depth_height:=400 \
  depth_fps:=30 \
  depth_format:=Y12 \
  enable_ir:=false \
  enable_point_cloud:=false
```

这里使用 `Y12` 是因为它是当前 Gemini Max 实际支持的深度格式。launch 文件中的默认值不一定适合每一款相机，运行时打印的设备 profile 才是判断支持格式的直接依据。

如果相机驱动已经在运行，则不需要重复启动。

4.3 检查深度话题是否存在

```
ros2 topic list
```

作用：列出当前 ROS 2 通信图中发现的话题。应当看到：

```
/camera/depth/image_raw
```

4.4 查询话题的消息类型

格式：

```
ros2 topic type <话题名>
```

本次命令：



```
ros2 topic type /camera/depth/image_raw
```

预期结果：



```
sensor_msgs/msg/Image
```

这决定了 Python 订阅者必须使用 `sensor_msgs.msg.Image` 类型。

4.5 查看一帧原始消息

格式：



```
ros2 topic echo <话题名> --once
```

本次命令：



```
ros2 topic echo /camera/depth/image_raw --once
```

重点关注：



```
height: 480
width: 640
encoding: 16UC1
step: 1280
```

16UC1 的含义：

- **16U**：每个数是 16 位无符号整数。
- **C1**：每个像素只有一个通道。
- 每个像素的数值代表深度，本驱动发布的数据单位为毫米。
- 数值 **0** 表示该像素没有有效深度，而不是距离为 0 mm。

step=1280 表示一行占用 1280 字节。因为宽度为 640，每个像素占 2 字节，所以 **640×2=1280**。

4.6 测量话题发布频率

格式：



```
ros2 topic hz <话题名>
```

本次命令：



```
ros2 topic hz /camera/depth/image_raw
```

作用：检查相机是否持续发布数据，以及实际频率是否接近配置的 30 Hz。

按 **Ctrl+C** 停止测量。

4.7 启动自己编写的节点

`ros2 run` 的标准格式是：



```
ros2 run <功能包名> <可执行程序名>
```

本次命令：



```
ros2 run depth_demo depth_reader
```

注意：这里最后一个参数是 `setup.py` 中注册的可执行程序名，不是 Python 文件的任意路径。

4.8 查看节点

列出节点：



```
ros2 node list
```

查看节点通信信息：



```
ros2 node info /depth_reader
```

本次结果中最关键的是：



```
Subscribers:  
/camera/depth/image_raw: sensor_msgs/msg/Image
```

它证明 `/depth_reader` 已经创建了对应订阅者。但是，`node info` 只能证明通信关系存在，不能单独证明消息一定已经到达。持续打印出的距离结果才证明回调函数确实收到了并处理了图像。

4.9 查看功能包注册的可执行程序



```
ros2 pkg executables depth_demo
```

预期看到类似：



```
depth_demo depth_reader
```

4.10 查看消息结构



```
ros2 interface show sensor_msgs/msg/Image
```

作用：查看 `Image` 消息包含哪些字段，例如时间戳、坐标系名称、宽高、编码格式和像素数据。

5. 创建 Python 功能包

进入工作空间的源码目录：

```
● ● ●
cd ~/orbbec_ros2_ws/src
```

正确的创建命令为：

```
● ● ●
ros2 pkg create depth_demo \
  --build-type ament_python \
  --license Apache-2.0 \
  --dependencies rclpy sensor_msgs cv_bridge python3-numpy
```

参数含义：

参数	作用
<code>depth_demo</code>	必需的位置参数，指定功能包名称
<code>--build-type ament_python</code>	创建 ROS 2 Python 功能包
<code>--license Apache-2.0</code>	写入许可证声明
<code>--dependencies ...</code>	声明程序所需依赖

本次曾使用以下错误命令：

```
● ● ●
ros2 pkg create --build-type ament_python --license Apache-2.0 \
  --dependencies rclpy sensor_msgs cv_bridge python3-numpy depth_demo
```

报错：

```
● ● ●
ros2 pkg create: error: the following arguments are required: package_name
```

原因是 `--dependencies` 会继续读取后面的内容，因此 `depth_demo` 被当成了一个依赖，而没有被识别为必需的功能包名称。

解决方法是把功能包名放在 `ros2 pkg create` 后面、`--dependencies` 前面。

另一个尝试是：

```
● ● ●
ros2 pkg create ... --node-name depth_demo
```

这同样不能代替功能包名。`--node-name` 只是要求工具额外生成一个节点模板，功能包名称仍然是必需的位置参数。

6. 功能包结构与入口配置

核心结构如下：

```

depth_demo/
├── package.xml
├── resource/
│   └── depth_demo
├── setup.cfg
├── setup.py
├── depth_demo/
│   ├── __init__.py
│   └── depth_reader.py
└── test/

```

`setup.py` 中需要包含可执行程序入口：

```

entry_points={
    'console_scripts': [
        'depth_reader = depth_demo.depth_reader:main',
    ],
},

```

这行的含义是：

```

ros2 run 使用的名字 = Python包.Python模块:入口函数
depth_reader          = depth_demo.depth_reader:main

```

如果没有配置或没有重新编译， `ros2 run depth_demo depth_reader` 就找不到可执行程序。

7. 最终版 `depth_reader.py`

```

import numpy as np
import rclpy
from cv_bridge import CvBridge
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from sensor_msgs.msg import Image

class DepthReader(Node):
    def __init__(self):
        super().__init__('depth_reader')

        self.bridge = CvBridge()
        self.frame_count = 0

        self.subscription = self.create_subscription(
            Image,
            '/camera/depth/image_raw',
            self.depth_callback,
            qos_profile_sensor_data,
        )

        self.get_logger().info(
            'Depth reader started. '
            'Waiting for /camera/depth/image_raw topic ...'
        )

```

```

def depth_callback(self, msg):
    self.frame_count += 1

    if msg.encoding != '16UC1':
        if self.frame_count % 30 == 0:
            self.get_logger().warning(
                f'Expected 16UC1, but received {msg.encoding}'
            )
        return

    try:
        depth_image = self.bridge.imgmsg_to_cv2(
            msg,
            desired_encoding='passthrough',
        )
    except Exception as error:
        self.get_logger().error(
            f'Failed to convert depth image: {error}'
        )
        return

    height, width = depth_image.shape
    center_x = width // 2
    center_y = height // 2

    radius = 5
    x_min = max(0, center_x - radius)
    x_max = min(width, center_x + radius + 1)
    y_min = max(0, center_y - radius)
    y_max = min(height, center_y + radius + 1)

    center_region = depth_image[y_min:y_max, x_min:x_max]
    valid_depths = center_region[center_region > 0]

    # 相机约为 30 FPS, 每 15 帧输出一次, 即约每秒输出两次。
    if self.frame_count % 15 != 0:
        return

    if valid_depths.size == 0:
        self.get_logger().info(
            'No valid depth near the image center'
        )
        return

    depth_mm = float(np.median(valid_depths))
    depth_m = depth_mm / 1000.0

    self.get_logger().info(
        f'Center=({center_x}, {center_y}), '
        f'valid_pixels={valid_depths.size}, '
        f'distance={depth_mm:.0f} mm ({depth_m:.3f} m)'
    )

```

```

def main(args=None):
    rclpy.init(args=args)
    node = DepthReader()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()

```

```
# Humble 收到 Ctrl+C 时可能已经关闭上下文，避免重复关闭。
if rclpy.ok():
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

注意：Markdown 中用于展示代码的 `python`` 和 ``` 不能复制到 `.py` 文件内部。

8. 程序逻辑逐步解释

8.1 导入依赖

```
import numpy as np
import rclpy
from cv_bridge import CvBridge
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from sensor_msgs.msg import Image
```

- `rclpy`：ROS 2 的 Python 客户端库。
- `Node`：用于创建 ROS 2 节点。
- `Image`：ROS 2 标准图像消息类型。
- `CvBridge`：在 ROS 图像消息和 OpenCV/NumPy 图像之间转换。
- `numpy`：完成切片、筛选和中位数计算。
- `qos_profile_sensor_data`：适合传感器数据的低延迟 QoS 配置。

8.2 创建节点

```
super().__init__('depth_reader')
```

这会让节点以 `/depth_reader` 的名称加入 ROS 2 通信图。

8.3 创建订阅者

```
self.subscription = self.create_subscription(
    Image,
    '/camera/depth/image_raw',
    self.depth_callback,
    qos_profile_sensor_data,
)
```

四个参数依次表示：

1. 消息类型是 `Image`。
2. 订阅 `/camera/depth/image_raw`。
3. 每收到一帧就调用 `depth_callback`。
4. 使用适合传感器数据的 QoS。

将订阅对象保存为成员变量可以避免它被 Python 垃圾回收。

8.4 为什么使用传感器 QoS

图像是高频、连续的传感器数据。对于实时感知，通常更关心最新一帧，而不是等待重传已经过时的旧帧。`qos_profile_sensor_data` 一般采用适合这类场景的策略，也更容易与相机驱动的 QoS 匹配。

如果发布者和订阅者的 QoS 不兼容，可能出现“话题存在，但订阅节点收不到数据”的情况。

8.5 检查深度编码

```
if msg.encoding != '16UC1':  
    ...
```

程序按照 16 位单通道无符号整数解释深度。如果实际编码不同，继续计算可能得到错误结果，所以先进行检查。

8.6 转换图像

```
depth_image = self.bridge.imgmsg_to_cv2(  
    msg,  
    desired_encoding='passthrough',  
)
```

`passthrough` 表示保留原来的 `16UC1` 编码，不把深度图转换成普通的 8 位灰度图。转换后，`depth_image` 是一个二维 NumPy 数组，每个元素对应一个像素的深度值。

8.7 获取图像中心

```
height, width = depth_image.shape  
center_x = width // 2  
center_y = height // 2
```

对于 `640×480` 图像：

```
center_x = 640 // 2 = 320  
center_y = 480 // 2 = 240
```

NumPy 图像访问顺序是 `[行, 列]`，也就是 `[y, x]`，而不是 `[x, y]`。

8.8 截取中心 11×11 区域

```
radius = 5  
center_region = depth_image[y_min:y_max, x_min:x_max]
```

中心点前后各取 5 个像素，加上中心自身，共得到：

```
(5 + 1 + 5) × (5 + 1 + 5) = 11 × 11 = 121 个像素
```

上限使用 `+1` 是因为 NumPy 切片包含起点但不包含终点。

与只读取一个中心像素相比，读取一个小区域可以降低单个坏点、噪声或缺失值造成的影响。

8.9 过滤无效深度

```
valid_depths = center_region[center_region > 0]
```

深度值 `0` 表示相机没有得到有效测量，常见原因包括：

- 物体过近或过远。
- 黑色、透明、镜面或强反光材质。
- 红外光受到太阳光等强光干扰。
- 物体边缘产生匹配失败。
- 相机刚启动，深度流尚未稳定。

因此，不能把 `0` 加入距离统计。

8.10 使用中位数

```
depth_mm = float(np.median(valid_depths))
```

中位数是排序后位于中间的数。它比平均值更不容易被少量异常大值或异常小值影响，因此适合基础深度测量。

8.11 单位转换

```
depth_m = depth_mm / 1000.0
```

本驱动输出的深度单位是毫米，因此除以 1000 得到米。

8.12 为什么每 15 帧输出一次

```
if self.frame_count % 15 != 0:  
    return
```

相机大约发布 30 FPS。如果每帧都打印，终端会每秒输出约 30 行，既不便观察，也会增加不必要的 I/O 开销。

每 15 帧打印一次意味着：

```
30 帧/秒 ÷ 15 帧/次 = 约 2 次/秒
```

因此日志每秒只有约两行不代表程序只处理 2 FPS。回调仍然接收每一帧，只是降低了打印频率。

9. 本次遇到的代码错误及解决方法

9.1 NumPy 切片使用了圆括号

错误代码：

```
center_region = depth_image(y_min:y_max, x_min:x_max)
```

报错：

```
SyntaxError: invalid syntax
```

原因：函数调用使用圆括号，而 NumPy 数组索引和切片必须使用方括号。

正确代码：

```
center_region = depth_image[y_min:y_max, x_min:x_max]
```

9.2 判断相等时使用了赋值符号

错误代码：

```
if valid_depths.size = 0:
```

原因：`=` 是赋值，`==` 才是比较是否相等。

正确代码：

```
if valid_depths.size == 0:
```

9.3 变量名拼写不一致

错误代码：

```
valid_depths = center_reagion[center_region > 0]
```

`center_reagion` 拼写错误，此变量不存在，会触发 `NameError`。

正确代码：

```
valid_depths = center_region[center_region > 0]
```

9.4 捕获异常和输出异常使用了不同变量名

错误代码：



```
except Exception as e:
    self.get_logger().error(f'Failed to convert depth image: {error}')
```

捕获的变量叫 `e`，输出时却使用了不存在的 `error`。

以下两种写法均可，但前后必须一致：



```
except Exception as error:
    self.get_logger().error(f'Failed to convert depth image: {error}')
```

9.5 QoS 名称没有正确导入

程序使用了：



```
qos_profile_sensor_data
```

因此必须导入：



```
from rclpy.qos import qos_profile_sensor_data
```

仅导入 `QoSProfile` 并不会自动产生名为 `qos_profile_sensor_data` 的变量。

9.6 区域上边界少了 +1

错误代码：



```
y_max = min(height, center_y + radius)
```

NumPy 切片不包含结束位置，因此应写为：



```
y_max = min(height, center_y + radius + 1)
```

否则纵向只能取得 10 个像素，而不是预期的 11 个像素。

9.7 订阅变量拼写错误

原代码写成了：



```
self.subsription = ...
```

这不一定立即导致程序失败，因为它仍然是一个成员变量，但拼写不规范容易在后续引用时引发问题。应统一为：



```
self.subscription = ...
```

9.8 Python 主入口写法错误

正确写法必须是：

● ● ●

```
if __name__ == '__main__':  
    main()
```

`__name__` 和 `'__main__'` 两侧都是双下划线。它表示只有直接运行该模块时才调用 `main()`。

9.9 Python 缩进或 Markdown 标记进入源码

Python 使用缩进表示代码层级。`class`、函数、`try`、`if` 内部必须保持一致缩进，一般使用 4 个空格。

聊天或 Markdown 中用于包围代码的三个反引号，以及后面可能跟随的 `python` 标记，只用于排版展示，不能写入 `.py` 文件。

9.10 启动时提示中心没有有效深度

输出：

● ● ●

```
No valid depth near the image center
```

这不是 Python 异常，而是程序按设计输出的状态信息。它表示当前中心 `11×11` 区域中的深度值全部为 `0`。

本次运行开始时出现了若干次该提示，随后输出：

● ● ●

```
valid_pixels=121, distance=965 mm
```

这说明程序和订阅始终正常，早期只是相机尚未在中心区域得到有效深度。

处理方法：

1. 等待相机启动并稳定数秒。
2. 在画面中心放置不透明、非镜面的物体。
3. 建议先放在约 `0.5~1.5 m` 范围。
4. 避免镜面、玻璃、纯黑吸光物体和强烈阳光直射。

9.11 按 `Ctrl+C` 后出现重复关闭错误

报错：

● ● ●

```
RCLError: failed to shutdown: rcl_shutdown already called on the given context
```

原因：在 ROS 2 Humble 中，按下 `Ctrl+C` 时，信号处理逻辑可能已经关闭 ROS 上下文，而程序的 `finally` 又无条件调用了一次 `rclpy.shutdown()`。

解决方法：

● ● ●

```
if rclpy.ok():  
    rclpy.shutdown()
```

这会先判断 ROS 上下文是否仍处于运行状态，从而避免重复关闭。

该错误只发生在退出阶段，不影响之前已经完成的深度测量。

9.12 日志字段连在一起

原输出类似：

```
Center = (320, 240)valid_pixels = 121distance = 965 mm
```

原因是多个相邻的 f-string 会自动拼接，但字符串中没有添加空格或逗号。

改进写法：

```
self.get_logger().info(
    f'Center=({center_x}, {center_y}), '
    f'valid_pixels={valid_depths.size}, '
    f'distance={depth_mm:.0f} mm ({depth_m:.3f} m)'
)
```

10. 语法检查、编译与运行

10.1 先做 Python 语法检查

```
cd ~/orbbec_ros2_ws/src/depth_demo
python3 -m py_compile depth_demo/depth_reader.py
```

如果没有任何输出，表示 Python 语法检查通过。

如果提示具体行号，可以使用以下命令带行号查看源码：

```
nl -ba depth_demo/depth_reader.py
```

10.2 编译指定功能包

```
cd ~/orbbec_ros2_ws
colcon build --packages-select depth_demo --symlink-install
```

说明：

- `colcon build`：构建 ROS 2 工作空间。
- `--packages-select depth_demo`：只构建当前 Demo，节省时间。
- `--symlink-install`：Python 源码通过符号链接安装，后续只修改 `.py` 文件时通常不必反复复制文件；但修改 `setup.py`、入口配置或依赖后仍应重新构建。

`colcon` 是 ROS 2 常用构建工具，但它不属于 `ros2 ...` 这套 CLI 子命令。

10.3 重新加载环境



```
source /opt/ros/humble/setup.bash
source ~/orbbeo_ros2_ws/install/setup.bash
```

编译后重新 `source` 的目的是让当前终端认识最新安装的功能包和可执行程序。

10.4 运行

确保另一个终端中的相机驱动正在发布深度图，然后执行：



```
ros2 run depth_demo depth_reader
```

按 `Ctrl+C` 正常停止。

11. 如何判断 Demo 是否真正成功

不能只凭“程序没有报错”判断成功，应分层验证。

第 1 层：驱动是否发布话题



```
ros2 topic list
```

应存在 `/camera/depth/image_raw`。

第 2 层：是否持续发布数据



```
ros2 topic hz /camera/depth/image_raw
```

应持续出现频率统计，通常接近 30 Hz。

第 3 层：Demo 是否创建订阅



```
ros2 node info /depth_reader
```

应显示它订阅 `/camera/depth/image_raw`。

第 4 层：回调是否收到并处理数据

程序应持续输出中心距离，例如：



```
Center=(320, 240), valid_pixels=121, distance=965 mm (0.965 m)
```

本次四层验证均已通过，因此可以确认基础深度 Demo 已经完成。

12. 对本次测量结果的理解

本次距离主要在 965~968 mm 之间变化，波动约 3 mm。这种小幅变化通常来自：

- 深度传感器本身的测量噪声。
- 物体表面反射特性。
- 红外散斑匹配误差。
- 相机与物体的轻微振动。
- 深度量化精度。

这不代表代码计算不稳定。相反，中心区域 121 个像素均有效，并且中位数只在几毫米范围内变化，说明当前测量较稳定。

还要注意：这里得到的是目标点沿相机光轴方向的深度值 z ，不一定等于该点到相机光心的三维欧氏距离。只有中心点附近二者非常接近。对于图像边缘，需要结合相机内参计算三维坐标后才能得到真正的空间直线距离。

13. 本次掌握的核心知识

1. ROS 2 节点可以作为数据生产者或消费者协同工作。
2. 相机驱动发布数据，算法节点订阅数据，显示工具也是独立订阅者。
3. ROS 2 图像通过 `sensor_msgs/msg/Image` 传输。
4. `16UC1` 深度图的每个像素是一个 16 位无符号深度值。
5. `CvBridge` 负责将 ROS 图像消息转换为 NumPy/OpenCV 图像。
6. NumPy 图像索引顺序是 `[y, x]`，切片必须使用方括号。
7. 深度值 0 表示无效测量，必须过滤。
8. 小区域中位数比单像素或简单平均值更抗异常点。
9. QoS 必须与传感器发布端兼容，否则可能有话题却收不到数据。
10. ROS 2 节点存在、订阅关系存在和数据真正到达是三个不同层次，需要分别验证。
11. 终端日志频率不一定等于相机帧率，本程序主动降低了打印频率。
12. 深度 z 与三维空间距离是两个不同概念。

14. 后续可扩展方向

完成这个 Demo 后，可以按以下顺序继续学习：

1. 在彩色图中心绘制十字和深度文字。
2. 同时订阅彩色图和深度图。
3. 理解彩色图与深度图的空间对齐，即 `depth_registration`。
4. 学习消息时间同步，避免使用不同时间拍摄的彩色帧和深度帧。
5. 订阅 `CameraInfo`，读取相机内参 `fx`、`fy`、`cx`、`cy`。
6. 将像素 (u, v) 和深度 z 转换为三维坐标 (X, Y, Z) ：

$$\begin{aligned} X &= (u - cx) \times Z / fx \\ Y &= (v - cy) \times Z / fy \\ Z &= \text{深度值} \end{aligned}$$

7. 从“测量画面中心距离”扩展到“鼠标点击任意像素测距”。

8. 结合目标检测，输出目标中心点的三维位置。
9. 将三维位置提供给机械臂抓取、导航或具身智能感知模块。

本次 Demo 是后续 RGB-D 感知的最小基础：它已经完成从真实传感器数据到可计算距离信息的完整闭环。